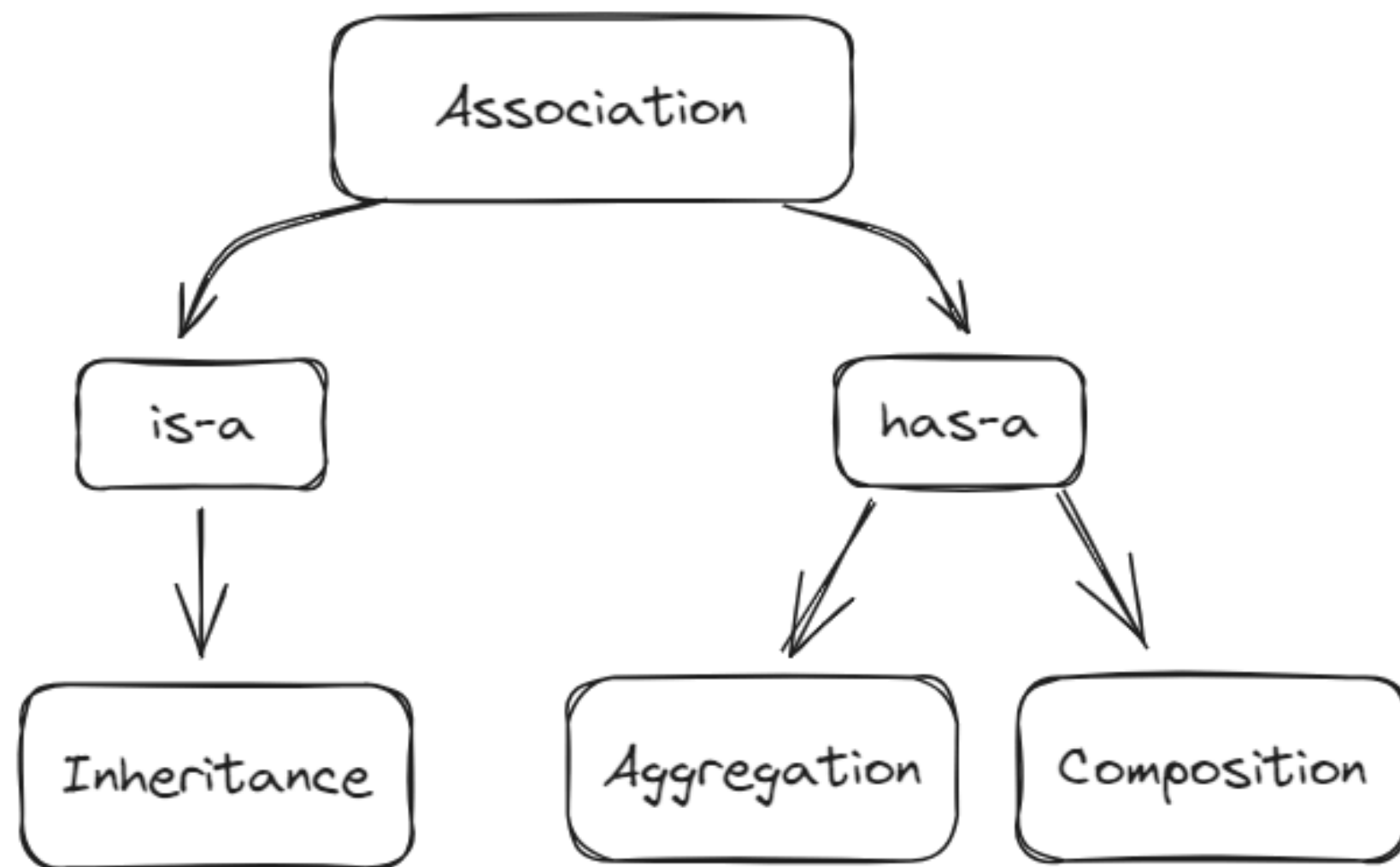


OOP Class Relationships

ASSOCIATION · AGGREGATION ·
COMPOSITION

How classes connect, own, and
depend on each other.





UML Notations



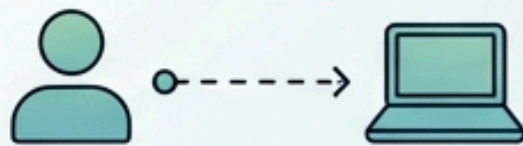
CLASSIFICATION OF OBJECT RELATIONSHIPS

TYPES OF RELATIONSHIPS

ASSOCIATION

→ **“USES-A”** (LOOSEST LINK)

A GENERAL RELATIONSHIP WHERE ONE OBJECT USES ANOTHER, NO OWNERSHIP.



EG: USER USES LAPTOP

AGGREGATION

→ **“HAS-A”** (CHILD SURVIVES ALONE)

A WEAK OWNERSHIP, WHERE THE CHILD OBJECT HAS ITS OWN LIFECYCLE.



EG: CAR HAS WHEELS
(WHEELS EXIST SEPARATELY)

COMPOSITION

→ **“PART-OF”** (CHILD DIES WITH PARENT)

A STRONG OWNERSHIP, CHILD CANNOT EXIST WITHOUT PARENT.



EG: BUILDING HAS ROOMS
(ROOMS ARE PART-OF BUILDING)

Association

- Definition: A general link between two independent classes
- Key point: No ownership — they just "know" each other
- Example: Student ↔ Instructor (a student learns from an instructor, but both exist independently)
- UML: plain line between classes

```
class Doctor {
public:
    string name;
    Doctor(string n) : name(n) {}

    void checkPatient(Stethoscope& s) { // ← Line B
        cout << name << " is using " << s.brand
            << " stethoscope." << endl;
    }
};

class Stethoscope { // ← Line A
public:
    string brand;
    Stethoscope(string b) : brand(b) {}
};

int main() {
    Stethoscope s1("Littmann"); // ← Line C
    Doctor d1("Dr. Sara"); // ← Line D

    d1.checkPatient(s1); // ← Line E

    // Both still exist here – no ownership
    return 0;
}
```



```

class Team {
public:
    string teamName;
    vector<Player*> players;

    Team(string t) : teamName(t) {}

    void addPlayer(Player* p) {
        players.push_back(p);
    }
};

class Player {
public:
    string name;
    Player(string n) : name(n) {}
};

int main() {
    Player p1("Ahmed");
    Player p2("Bilal");
    Player p3("Zara");

    {
        Team t("Lions");
        t.addPlayer(&p1);
        t.addPlayer(&p2);
        t.addPlayer(&p3);
    }
    return 0;
}

```

Aggregation

- Definition: "Has-a" but the child can live without the parent
- Key point: Weak ownership — shared, not exclusive
- Example: Department has Teachers — but if Department is deleted, Teachers still exist
- UML: hollow diamond on parent side

Composition

- Definition: "Part-of" — child cannot exist without parent
- Key point: Strong ownership — lifecycle is shared
- Example: House has Rooms — destroy the House, Rooms are gone too
- UML: filled/solid diamond on parent side

```
class Car {
public:
    string model;
    Engine engine;           // ← Line C (by value, not pointer!)

    Car(string m, int hp)
        : model(m), engine(hp) { // ← Line D
        cout << "Car created: " << model << endl;
    }
    void showInfo() {
        cout << model << " has "
             << engine.horsepower << " HP engine." << endl;
    }
};

class Engine {               // ← Line A
public:
    int horsepower;
    Engine(int hp) : horsepower(hp) {
        cout << "Engine created (" << hp << " HP)" << endl;
    }
};

int main() {
    {
        Car c("Toyota", 150);   // ← Line E
        c.showInfo();
    }                           // ← Line F: Car AND Engine both destroyed
    return 0;
}
```

THANK YOU